

Bitwise Operators in Java

 [geeksforgeeks.org/bitwise-operators-in-java](https://www.geeksforgeeks.org/bitwise-operators-in-java)

August 8, 2018

Bitwise Operators

Bitwise operators are used to performing the manipulation of individual bits of a number. They can be used with any integral type (char, short, int, etc.). They are used when performing update and query operations of the Binary indexed trees.

Now let's look at each one of the bitwise operators in Java:

1. Bitwise OR (|)

This operator is a binary operator, denoted by '|'. It returns bit by bit OR of input values, i.e., if either of the bits is 1, it gives 1, else it shows 0.

Example:

```
a = 5 = 0101 (In Binary)
b = 7 = 0111 (In Binary)
```

Bitwise OR Operation of 5 and 7

```
  0101
| 0111
```

```
  0111 = 7 (In decimal)
```

2. Bitwise AND (&)

This operator is a binary operator, denoted by '&.' It returns bit by bit AND of input values, i.e., if both bits are 1, it gives 1, else it shows 0.

Example:

```
a = 5 = 0101 (In Binary)
b = 7 = 0111 (In Binary)
```

Bitwise AND Operation of 5 and 7

```
  0101
& 0111
```

```
  0101 = 5 (In decimal)
```

3. Bitwise XOR (

This operator is a binary operator, denoted by '^.' It returns bit by bit XOR of input values, i.e., if corresponding bits are different, it gives 1, else it shows 0.

Example:

a = 5 = 0101 (In Binary)
b = 7 = 0111 (In Binary)

Bitwise XOR Operation of 5 and 7

```
  0101
^ 0111
-----
 0010 = 2 (In decimal)
```

4. Bitwise Complement (~)

This operator is a unary operator, denoted by '~.' It returns the one's complement representation of the input value, i.e., with all bits inverted, which means it makes every 0 to 1, and every 1 to 0.

Example:

a = 5 = 0101 (In Binary)

Bitwise Complement Operation of 5

```
~ 0101
-----
 1010 = 10 (In decimal)
```

Note: Compiler will give 2's complement of that number, i.e., 2's complement of 10 will be -6.

Advantages

The advantages of using Bitwise Operators in Java are:

1. **Speed:** Bitwise operations are much faster than arithmetic operations as they operate directly on binary representations of numbers.
2. **Space Optimization:** Bitwise operations can be used to store multiple values in a single variable, which can be useful when working with limited memory.
3. **Bit Manipulation:** Bitwise operators allow for precise control over individual bits of a number, which can be useful in various applications such as cryptography, error detection, and compression.
4. **Code Simplification:** Bitwise operations can simplify the code by reducing the number of conditional statements and loops required to perform certain tasks.
5. **Improved Readability:** Bitwise operations can make the code more readable by encapsulating complex logic into a single operation, making the code easier to understand and maintain.